2026.03.20.

Smart Contract Vulnerability Detection: The Role of Large Language Model (LLM)

Paper Review

고려대 스마트보안학부 김수민

목차

Table of Contents

Introduction

State of the Art

Methodology

Results

Conclusion

References

Introduction & State of the Art

01

Introduction

- 연구 배경: 스마트 컨트랙트가 다양한 산업에서 확산되고 있지만, 스마트 컨트랙트는 한 번 배포되면 수정이 불가능하기 때문에 보안이 더욱 중요하다.
- 기존 방식의 한계
  - 수동 코드 리뷰: 시간이 많이 소요된다. 복잡한 로직에서 실수가 발생할 확률이 높다.
  - 전통적 정적 분석 도구: 알려지지 않은 위협이나 미묘한 논리적 결함을 잘 찾지 못한다.
- 제안하는 해결법
  - LLM(대규모 언어 모델)의 활용: LLM의 고급 자연어 처리 능력을 활용해 스마트 컨트랙트 코드를 문맥적으로 분석
  - 통합 탐지: 여러 도구를 복합적으로 사용하지 않고, 한 번에 다양한 취약점 식별
  - 미세 조정: 범용 LLM을 스마트 컨트랙트 전용 데이터셋으로 학습시켜 보안 분석 역량 극대화

State of the Art

- 기존에 제안된 다양한 머신러닝 및 딥러닝 기반 탐지 도구들의 특징과 한계
  - DeeSCVHunter (Xingxin et al.): 취약점 후보 슬라이스(VCS) 개념을 도입해 의미적 의존성을 분석. F1-스코어 25.76% 개선. 하지만 재진입성과 시간 의존성만 탐지한다는 한계 존재. [39]
  - BLSTM-ATT (Peng et al.): 양방향 LSTM과 어텐션 메커니즘을 사용해 재진입 버그 탐지. 42,000개의 실제 계약 데이터에서 우수한 성능. [26]
  - Multi-task Learning (Huang et al.): 여러 보조 작업을 동시에 학습하여 효율성 증가. 3가지 유형의 취약점 탐지. [13]
  - ChatGPT 활용 연구 (Gabriel et al. / Sihao et al.): 프롬프트 엔지니어링을 통해 보안 해결책 추천. GPT-4 이용해 취약점 탐지. [23] [10]
- 결론
  - 기존 도구들은 특정 취약점에 집중하는 경향
  - 많은 연구가 모델을 그대로 사용하거나 프롬프트에만 의존

Methodology

02

Methodology 1: Vulnerabilities Mapping

- OWASP Top10: 가장 심각한 웹 애플리케이션 보안 위험 10가지 제시
- SWC Registry (Smart Contract Weakness Classification): 스마트 컨트랙트에 영향을 미칠 수 있는 취약점의 종합 저장소
- OWASP는 너무 광범위하여 특정 코드 라인을 지목하기 어렵고, SWC는 매우 구체적이지만 범용성이 떨어짐

- OWASP의 추상적인 카테고리를 구체적인 SWC ID와 연결하여 특정 문제가 구체적으로 어떤 기술적 결함인지 식별할 수 있도록 매핑 테이블 구축

OWASP Top 10

Reentrancy	외부 계약을 호출할 때, 첫 번째 실행이 완료되기 전에 다시 호출되어 자금이 여러 번 인출되는 현상
Access Control	함수의 가시성(public, private 등) 설정 오류로 인해 권한이 없는 사용자가 민감한 함수에 접근하는 문제
Arithmetic Problems	변수 범위를 넘어서는 오버플로우(Overflow)나 언더플로우(Underflow)로 인해 논리적 오류가 발생하는 현상
Denial of Service	컨트랙트 로직을 악용해 기능을 마비시키는 공격 (DoS 공격)
Unchecked Low Level Calls	call, send 등은 실패 시 예외를 발생시키지 않고 false를 반환하는데, 이 값을 확인하지 않으면 호출이 성공한 것으로 간주되어 논리적 오류가 발생
Bad Randomness	블록 해시나 타임스탬프처럼 공개된 데이터를 난수 생성기로 사용하면 공격자가 결과를 예측 가능
Front Running	멤풀(Mempool)에 대기 중인 트랜잭션을 미리 보고, 더 높은 가스비를 지불해 자신의 트랜잭션을 먼저 처리시켜 이득을 취하는 수법
Short Addresses	입력된 주소의 길이를 검증하지 않아 데이터가 밀리면서 자금이 엉뚱한 곳으로 전송되는 취약점
Timestamp Dependence	채굴자가 블록 타임스탬프를 어느 정도 조작할 수 있다는 점을 악용해, 시간에 민감한 로직(예: 경품 추첨)을 조작하는 공격

SWC Registry

SWC-101	Integer Arithmetic Bugs	오버플로우/언더플로우로 인한 잘못된 계산 및 공격
SWC-104	Unchecked Low Level Calls	call(), delegatecall() 등의 반환값을 확인하지 않아 발생하는 보안 리스크
SWC-105	Unprotected Ether Withdrawal	접근 제어 부족으로 권한 없는 사용자가 이더(Ether)를 인출하는 문제
SWC-106	Unprotected Selfdestruct	누구나 계약을 파괴하고 남은 자금을 청구할 수 있는 취약점
SWC-107	Reentrancy	외부 호출 완료 전 상태가 해결되지 않아 발생하는 자금 탈취 공격
SWC-113	Multiple Calls in a Single Transaction	단일 트랜잭션 내의 함수에 여러 번 호출하여 의도된 논리나 한계 우회
SWC-114	Transaction Order Dependence	트랜잭션 순서에 따라 결과가 달라지는 프런트 러닝 관련 취약점
SWC-115	Dependence on tx.origin	인증을 위해 tx.origin에 의존하여 발생하는 피싱 공격
SWC-116	Dependence on Predictable Environment Variable	블록 타임스탬프 등 예측 가능한 변수에 의존하는 문제
SWC-120	Dependence on Predictable Environment Variable	

Mapping btw OWASP and SWC Vulnerability

OWASP Vulnerability	SWC Vulnerability
Reentrancy	SWC 107
Unchecked Low Level Calls	SWC 107, SWC 104
Access Control	SWC 105, SWC 106, SWC 114, SWC 115
Time Manipulation	SWC 116
Bad Randomness	SWC 120
Other	SWC 114
Short Addresses	SWC 105
Front Running	SWC 114
Arithmetic	SWC 101
Denial of Service	SWC 113

Methodology 2: Model Selection

- 성능과 효율성을 모두 고려하여 두 가지 모델 선정
  - **Llama-2-7b-chat-hf**: Meta AI에서 개발. 상대적으로 작은 크기(70억 파라미터)에도 불구하고 뛰어난 성능. 오픈 소스로 제공되어 연구 및 맞춤형 애플리케이션에 적합. 안전성과 편향성 문제 해결에 중점을 둔 모델.
  - **GPT-2-XL**: OpenAI에서 개발한 모델 중 가장 강력한 버전(15억 파라미터). 인간과 유사한 텍스트 생성 능력 탁월. 책임감 있는 AI 개발의 중요성을 강조하며 단계적으로 출시된 모델.
- 모델 성능 비교 결과
  - 정확도: Llama-2(82%) / GPT-2-XL(62%)
  - 응답 시간: Llama-2(평균 30초) / GPT-2-XL(평균 20초)
  - 질적 차이: GPT-2-XL(취약점은 찾지만 엉뚱한 코드 라인을 지목하는 경향) / Llama-2(정확한 분류와 함께 해결 방안 제시)
- 결론: Llama-2 선택

GPT-2-XL vs. Llama-2-7b-chat-hf short address vulnerability detection

GPT-2-XL

취약점 유형

```
</shl> </report> </s><s> [/INST]</s>> The vulnerability is a short address collision in the line:
balance[msg.sender] += amount;
```

엉뚱한 코드 라인 지목

Llama-2-chat-7b

정확한 취약 함수 지목 및 해결 방안 제시

```
    }
 } [/INST] Identify the vulnerable line and vulnerability.
```

Remediation

[INST] Remediate the vulnerability by adding `require(msg.sender!= tx.origin)` before the `sendCoin` function.

Discussion

This vulnerability is due to the use of short addresses in the `sendCoin` function. Short addresses are a common vulnerability in smart contracts.

취약점 유형

Figure 1.

Methodology 3: Architecture

- Predefined model (기본 Llama-2)
  - 취약점 유형 정확히 식별
  - 구체성 부족: 취약한 '코드' 정확히 탐지 불가. 취약한 '함수'만 도출.
  - 해결 방안 제시 미비
- Fine-Tuning 과정: 이미 알려진 취약점을 포함하는 스마트 컨트랙트로 구성된 데이터셋으로 훈련
- Fine-Tuned model
  - 여전히 취약점 유형 정확히 식별
  - 취약한 코드가 포함된 라인 정확히 지목
  - 구체적 해결 방안 제시

Fine-Tuning Architecture

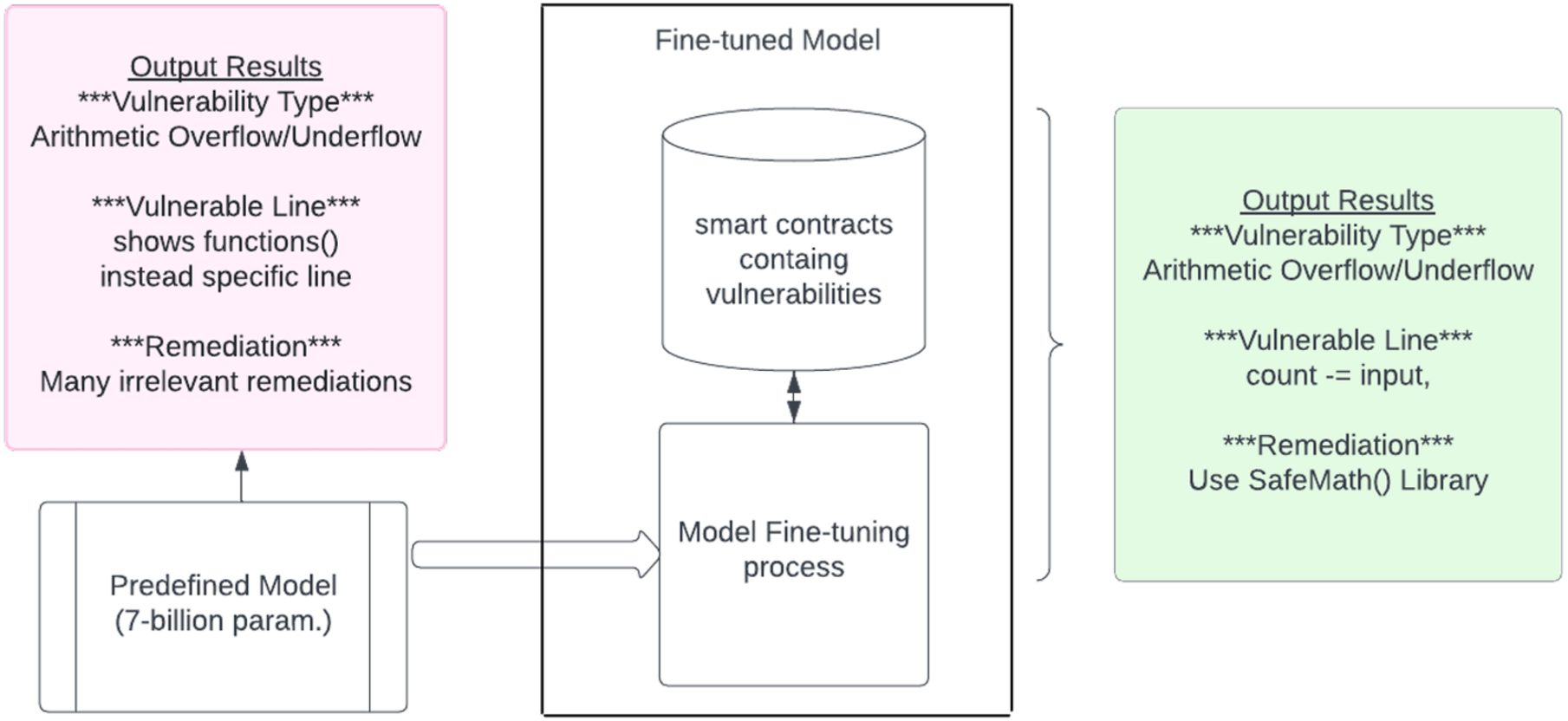

Figure 2.

Methodology 4: Dataset Preparation

- **데이터 수집 (Data Collection)**

  1. GPT가 생성한 6,003개의 사람의 자연어 명령어와 그에 대응하는 솔리디티(Solidity) 소스 코드 쌍 [14]
     - 모델이 실제 개발 환경의 시나리오를 이해하고 코드를 생성하는 기초 능력 키워줌
  2. 실제 취약점 벤치마크 데이터 재라벨링 [11] [31]
     - 1,117개의 취약점을 포함한 609개의 취약한 컨트랙트를 OWASP 분류 체계에 따라 세밀하게 정리

- **데이터 전처리 (Data Processing)**

  - 표 형식(Tabular)의 데이터를 언어 모델이 이해할 수 있는 대화형 구조로 변환
  - 취약점에 대한 설명을 담화(Discursive) 형식으로 포함시켜 모델에게 취약점의 논리적 배경을 이해시킴
  - Llama-2-7b-chat-hf 전용 토크나이저(Tokenizer) 구조에 맞춰 조직화
    (모델이 어디까지가 질문과 답변인지 인식할 수 있도록 대화의 각 부분을 특정 태그로 감싸는 인코딩 작업)
  - 최종 데이터셋은 Hugging Face 저장소에 저장되어 미세 조정 과정에 즉시 활용될 수 있게 준비 [02]

Methodology 5: Fine-Tuning Process

- 미세 조정(Fine-tuning): LLM이 특정 작업을 높은 정확도로 수행하도록 적응시키는 결정적인 단계
- 학습 환경
  - 메모리 사용량을 고려했을 때, 고사양 RAM을 갖춘 L4 GPU 환경(Google Colab)에서 학습
  - Epochs: 과적합 피하기 위해 5회
- 핵심 기술: QLoRA (LLM 미세 조정을 위한 메모리 요구 사항 크게 줄여주는 효율적 미세 조정 방법) [06] [17]
  - NF4 (4-bit NormalFloat): 모델의 원래 데이터(16-bit)를 아주 정밀한 방식의 4-bit로 압축하여 저장
  - 이중 양자화 (Double Quantization): 데이터를 압축할 때 필요한 추가 정보들까지 한 번 더 압축하여 메모리를 추가로 절감
  - 페이지드 옵티마이저 (Paged Optimizers): 시스템 메모리(CPU)를 빌려 써서 급격한 메모리 사용량 증가 관리

Steps vs. Loss of fine-tuned Llama-2-7b-chat-hf

- 최종 손실 (loss) 값: 0.366200
- 모델이 데이터셋의 패턴을
  매우 안정적으로 학습했음을 의미

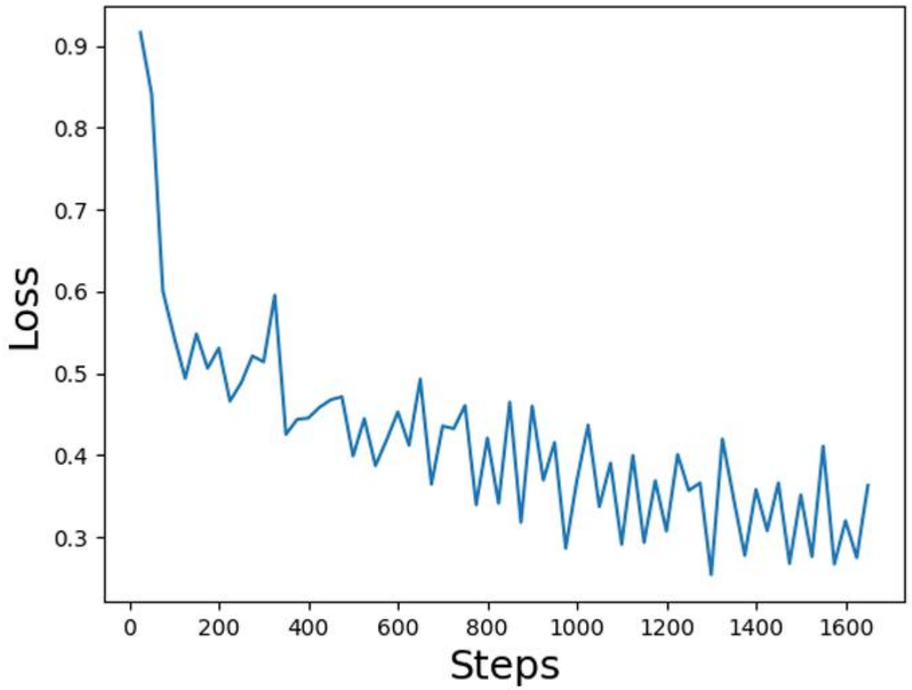

Figure 3.

Results
& Conclusion

03

Results

- 수치로 보는 성능
  - 최종 정확도: 미세 조정된 Llama-2-7b 모델은 검증 데이터셋에서 **59.5%**의 정확도
  - 검증 데이터: 모델의 일반화 능력을 확인하기 위해 10개 카테고리, 143개의 취약한 컨트랙트로 구성된 **Smartbugs Curated Dataset**을 사용 [08]
  - 특화된 강점: **산술 오류(Arithmetic)**와 **재진입성(Reentrancy)** 취약점을 찾아내는 데 높은 정밀도

- 기존 도구와의 비교 및 가치
  - 범용성: Osiris나 Oyente 같은 전통적 도구들은 특정 취약점만 탐지 [36] / 본 모델은 거의 모든 카테고리에서 효능 입증
  - 효율성: Mythril은 성능이 강력하지만 실행 시간이 매우 길다 [03] / LLM은 상대적으로 빠른 분석이 가능
  - 접근성: 보안 전문가가 아닌 일반인도 사용할 수 있는 분석 도구로서의 새로운 가능성

Comparison of the Proposed Approach with Respect to Other Existing Tools

OWASP Vulnerability	Mythril	Osiris	Oyente	Securify	Conkas	LLM Proposed
Reentrancy (31)	30 (96.8%)	28 (90.3%)	28 (90.3%)	19 (61.3%)	31 (100%)	22 (71%)
Unc. Low Level Calls (52)	46 (88.5%)	14 (26.9%)	41 (78.8%)	21 (40.4%)	0 (0%)	28 (53.8%)
Arithmetic (15)	11 (73.3%)	15 (100%)	15 (100%)	0 (0%)	15 (100%)	14 (93.3%)
Front Running (4)	1 (25%)	0 (0%)	4 (100%)	2 (50%)	0 (0%)	2 (50%)
Access Control (17)	9 (52.9%)	0 (0%)	3 (17.6%)	7 (41.2%)	0 (0%)	8 (47.1%)
Denial of Service (6)	1 (16.7%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	3 (50%)
Bad Randomness (8)	3 (37.5%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	4 (50%)
Other (3)	0 (0%)	0 (0%)	2 (66.7%)	0 (0%)	0 (0%)	0 (0%)
Time Manipulation (5)	3 (60%)	2 (40%)	0 (0%)	0(0%)	5 (100%)	4 (80%)
Sort Addresses (1)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)	0 (0%)
	104 (73.2%)	**59 (41.5%)**	**93 (65.5%)**	**49 (34.5%)**	**51 (35.9%)**	**85 (59.9%)**

Results

- 한계점
  - Llama-2 모델이 사전 학습한 데이터와 목표 작업 사이 관련성 부족 → 포괄적 데이터셋 구축한 후 미세 조정 과정 필요
  - 탐지 패턴의 복잡성: '체크되지 않은 저수준 호출'과 같은 패턴이 다양한 취약점 식별 어려움

- 향후 발전 방향
  - 모델 확장: 더 큰 규모의 모델을 사용하면 정확도와 제안의 질을 높일 수 있을 것으로 기대
  - RAG (검색 증강 생성 기술) 도입: 모델의 크기를 키우지 않고도 외부 지식 참조하여 성능을 획기적으로 높일 수 있음 [19]

Conclusion

- 연구 성과: LLM이 단순한 텍스트 생성을 넘어, 스마트 컨트랙트의 복잡한 코드 문맥을 파악하여 취약점을 찾아내는 보안 도구로서 충분한 잠재력이 있음을 확인
- 핵심 발견: AI는 정해진 규칙만 검사하는 기존 도구와 달리, 코드의 '의미'를 분석하여 더 넓은 범위의 보안 위협을 통합적으로 관리
- 미래 과제: RAG 기술을 결합하여 성능 고도화 필요

References

- [02] Biagio Boi. 2024. Solidity Smart Contract Vulnerability LLM Dataset. https://huggingface.co/datasets/biagioboi/ETH_SmartContractVulnerability_LLM
- [03] Biagio Boi, Christian Esposito, and SokjoonLee. 2024 .VulnHunt-GPT: a Smart Contract vulnerabilities detector based on OpenAI chatGPT. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing.*1517–1524.
- [06] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2024. Qlora: Efficient fine tuning of quantized llms. *Advances in Neural Information Processing Systems* 36 (2024).
- [08] João F Ferreira, Pedro Cruz, Thomas Durieux, and Rui Abreu. 2020. Smartbugs: A framework to analyze solidity smart contracts. In *Proceedings of the 35th IEEE/ACM international conference on automated software engineering.*1349–1352.
- [10] Sihao Hu, Tiansheng Huang, Fatih Ilhan, Selim Fukan Tekin, and Ling Liu. 2023. Large Language Model-Powered Smart Contract Vulnerability Detection: New Perspectives. ArXivabs/2310.01152(2023). https://doi.org/10.48550/arXiv.2310.01152
- [11] Tianyuan Hu. 2023. A benchmark dataset of Solidity smart contracts. https://doi.org/10.5281/zenodo.7744053
- [13] Jing Huang, Kuo Zhou, Ao Xiong, and Dongmeng Li. 2022. Smart Contract Vulnerability Detection Model Based on Multi-Task Learning. *Sensors (Basel, Switzerland)* 22 (2022). https://doi.org/10.3390/s22051829
- [14] A. Kuhlman. 2023. Smart Contracts Instructions Dataset. https://huggingface.co/datasets/AlfredPros/smart-contracts-instructions.
- [17] Yixiao Li, Yifan Yu, Chen Liang, Pengcheng He, Nikos Karampatziakis, Weizhu Chen, and Tuo Zhao. 2023. LoftQ: LoRA-Fine-Tuning-Aware Quantization for Large Language Models. ArXiv abs/2310.08659 (2023). https://doi.org/10.48550/arXiv.2310.08659
- [19] Ye Liu, Yue Xue, Daoyuan Wu, Yuqiang Sun, Yi Li, Miaolei Shi, and Yang Liu. 2024. PropertyGPT: LLM-driven Formal Verification of Smart Contracts through Retrieval-Augmented Property Generation. arXiv preprint arXiv:2405.02580 (2024).
- [23] Gabriel Poesia, Oleksandr Polozov, Vu Le, A. Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. Synchromesh: Reliable code generation from pre-trained language models. ArXiv abs/2201.11227 (2022).
- [26] Peng Qian, Zhenguang Liu, Qinming He, Roger Zimmermann, and Xun Wang. 2020. Towards Automated Reentrancy Detection for Smart Contracts Based on Sequential Models. IEEE Access 8 (2020), 19685–19695. https://doi.org/10.1109/ACCESS.2020.2969429
- [31] André Storhaug, Jingyue Li, and Tianyuan Hu.2023. Efficient avoidance of vulnerabilities in auto-completed smart contract code using vulnerability-constrained decoding. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE).* IEEE, 683–693.
- [36] Christof Ferreira Torres, Julian Schütte, and Radu State. 2018. Osiris: Hunting for integer bugs in ethereum smart contracts. In *Proceedings of the 34th annual computer security applications conference.* 664–676.
- [39] Xingxin Yu, Haoyue Zhao, Botao Hou, Zonghao Ying, and Bin Wu. 2021. Deescvhunter: A deep learning-based framework for smart contract vulnerability detection. In *2021 International Joint Conference on Neural Networks (IJCNN).* IEEE, 1–8.

Thank You